

Lecture 1.2: Geospatial Data Types and Python for GIS

From file formats to reproducible geospatial workflows

Dr. John C. Femiani

CSE620B: Remote Sensing and Computer Vision
Miami University

Course Map for Today

- 1 Why Python for Geospatial Work
- 2 Raster Data with rasterio
- 3 Vector Geometry with Shapely
- 4 CRS, EPSG, and Reprojection
- 5 Integrated Analysis Pipelines
- 6 Notebook Guidance and Deliverables

Roadmap Checkpoint

- 1 Why Python for Geospatial Work
- 2 Raster Data with rasterio
- 3 Vector Geometry with Shapely
- 4 CRS, EPSG, and Reprojection
- 5 Integrated Analysis Pipelines
- 6 Notebook Guidance and Deliverables

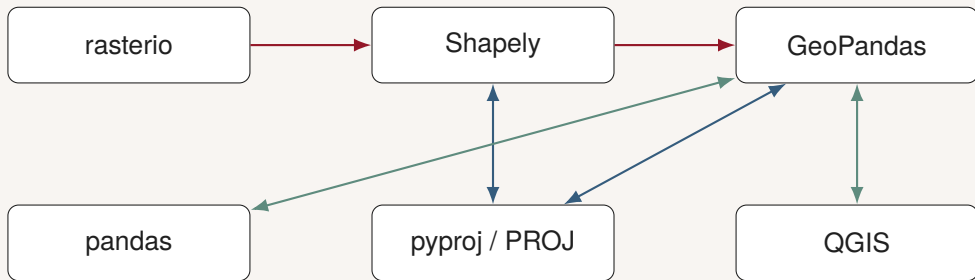
Learning Targets

- Read and inspect raster data with rasterio
- Build and query vector geometry with Shapely and GeoPandas
- Transform coordinates using EPSG-aware reprojection
- Combine raster and vector pipelines for analysis and mapping

Guiding idea

Python gives us one language for IO, geometry, projection, and analysis.

Core Python Stack for GIS



- Use each library where it is strongest, not one library for everything.

Raster vs Vector: Operational View

Raster

- Grid of cells
- Continuous fields: temperature, elevation, reflectance
- Band-aware matrix operations

Vector

- Points, lines, polygons
- Discrete features: roads, parcels, boundaries
- Topology and geometry relations

Workflow consequence

Most remote sensing projects require both models in the same pipeline.

Roadmap Checkpoint

- 1 Why Python for Geospatial Work
- 2 **Raster Data with rasterio**
- 3 Vector Geometry with Shapely
- 4 CRS, EPSG, and Reprojection
- 5 Integrated Analysis Pipelines
- 6 Notebook Guidance and Deliverables

NumPy Mental Model for Raster

- Single-band raster: matrix $R \in \mathbb{R}^{h \times w}$
- Multi-band raster: tensor $X \in \mathbb{R}^{b \times h \times w}$
- Pixel indexing: row-major (y first, then x)
- Metadata binds array values to real world coordinates

Practical check

Always inspect shape, dtype, transform, and CRS before analysis.

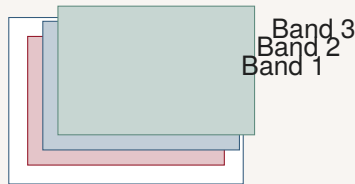
Inspect a Raster File

```
import rasterio

with rasterio.open("scene.tif") as src:
    print(src.count)          # number of bands
    print(src.shape)         # (rows, cols)
    print(src.crs)           # coordinate reference system
    print(src.transform)     # pixel to map affine transform
    print(src.nodata)       # missing-data sentinel
```

Understanding Raster Bands

- RGB imagery stores one channel per band
- Remote sensing scenes can have 4, 8, or 100+ bands
- A band is not always color; often it is wavelength-specific reflectance



NoData Handling

- NoData marks pixels with invalid or missing measurement
- Ignoring NoData can bias means, histograms, and classifiers
- Convert raw arrays to masked arrays before statistics

Safe pattern

Use mask-aware operations first; fill only at export time.

Masking NoData in Practice

```
import numpy as np
import rasterio

with rasterio.open("dem.tif") as src:
    band = src.read(1)
    nodata = src.nodata

masked = np.ma.masked_equal(band, nodata)
mean_elev = masked.mean()
```

Raster Operations You Use Weekly

- Windowed reads for large scenes
- Clipping by AOI polygon
- Reprojection to common analysis CRS
- Writing clean GeoTIFF outputs with metadata lineage

Deliverable quality

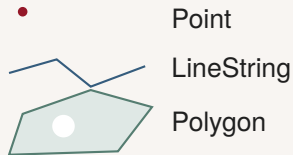
Persist CRS, transform, nodata, and dtype explicitly in outputs.

Roadmap Checkpoint

- 1 Why Python for Geospatial Work
- 2 Raster Data with rasterio
- 3 Vector Geometry with Shapely**
- 4 CRS, EPSG, and Reprojection
- 5 Integrated Analysis Pipelines
- 6 Notebook Guidance and Deliverables

Shapely Geometry Types

- Point: location sample
- LineString: path or boundary segment
- Polygon: area with optional interior holes
- Multi* collections for grouped features



Geometric Properties and Predicates

```
from shapely.geometry import Point, Polygon

p = Point(-84.73, 39.51)
aoi = Polygon([(-84.8, 39.45), (-84.6, 39.45),
               (-84.6, 39.6), (-84.8, 39.6)])

print(aoi.area)
print(aoi.bounds)
print(aoi.contains(p))
```


Core Set Operations

Operations

- Intersection
- Union
- Difference
- Buffer

Use cases

- Keep roads inside a county
- Merge adjacent land parcels
- Remove water from candidate land
- Build safety zones around schools

From Geometry to Geospatial Workflow

- Read features with GeoPandas or Fiona
- Apply Shapely operations on geometry column
- Preserve attributes while transforming shapes
- Export to GeoPackage, GeoJSON, or Shapefile

Common pitfall

Do not run metric operations until data is in a projected CRS with meter units.

Roadmap Checkpoint

- 1 Why Python for Geospatial Work
- 2 Raster Data with rasterio
- 3 Vector Geometry with Shapely
- 4 CRS, EPSG, and Reprojection**
- 5 Integrated Analysis Pipelines
- 6 Notebook Guidance and Deliverables

Why Projection Management Matters

- Earth is 3D; maps are 2D approximations
- Every projection trades off angle, area, distance, or direction
- Mixed-CRS layers are a top source of geospatial bugs

Rule

If two layers do not share CRS, transform one before overlay or distance.

EPSG Codes in Daily Work

- EPSG:4326: WGS84 longitude/latitude
- EPSG:3857: Web Mercator tiles
- UTM zones: local metric analysis

Quick mapping

Data collection: 4326

Web display: 3857

Distance/area stats: UTM

pyproj Transformer Pattern

```
from pyproj import Transformer

transformer = Transformer.from_crs("EPSG:4326", "EPSG:26916", always_xy=True)
x, y = transformer.transform(-84.73, 39.51)
print(x, y)
```

- Use `always_xy=True` to keep (lon, lat) input ordering explicit.

GeoPandas Reprojection Pattern

```
import geopandas as gpd

gdf = gpd.read_file("roads.geojson")
print(gdf.crs)

roads_utm = gdf.to_crs("EPSG:26916")
roads_utm["len_m"] = roads_utm.length
```

Proj vs Geod: Which Tool?

Task	Use	Typical output
Coordinate transform	Proj / Transformer	x,y in new CRS
Great-circle distance	Geod.inv	meters + azimuth
Forward geodesic point	Geod.fwd	destination lon,lat

Roadmap Checkpoint

- 1 Why Python for Geospatial Work
- 2 Raster Data with rasterio
- 3 Vector Geometry with Shapely
- 4 CRS, EPSG, and Reprojection
- 5 Integrated Analysis Pipelines**
- 6 Notebook Guidance and Deliverables

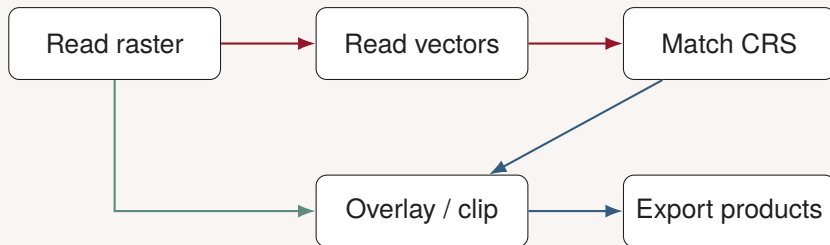
Crop Vector Features to Raster Extent

- 1 Open raster and get bounds in raster CRS
- 2 Convert bounds to a polygon geometry
- 3 Reproject vector layer into raster CRS
- 4 Clip vectors to raster bounds

Result

Aligned spatial support for training, evaluation, and map export.

Pipeline Blueprint: Raster + Vector + CRS



Visual Inspection in QGIS

- Open intermediate outputs to catch CRS mismatches early
- Check symbology, extents, and NoData transparency
- Validate geometry integrity before model training
- Keep a lightweight project file as a QA artifact

Review: pandas and GeoPandas Roles

pandas

- Tabular joins and cleaning
- Groupby and aggregation
- Time and metadata handling

GeoPandas

- Spatial join and clipping
- CRS-aware geometry column
- Direct plotting and file export

Roadmap Checkpoint

- 1 Why Python for Geospatial Work
- 2 Raster Data with rasterio
- 3 Vector Geometry with Shapely
- 4 CRS, EPSG, and Reprojection
- 5 Integrated Analysis Pipelines
- 6 Notebook Guidance and Deliverables**

Practice Sequence for This Lecture

- 1 Read slides once end-to-end
- 2 Run raster notebook: inspect bands, metadata, NoData
- 3 Run vector notebook: geometry ops and OpenStreetMap data
- 4 Verify one output from each notebook in QGIS

Submission habit

Save notebooks with clear markdown notes about CRS choices and assumptions.

Troubleshooting Checklist

- Wrong extents: inspect CRS and transform
- Empty overlay result: reproject before clipping
- Impossible distances: check projected units
- Jagged masks: verify raster resolution and resampling mode

Key Takeaways

- Data model drives method: raster for fields, vector for objects
- CRS consistency is a requirement, not an optional cleanup step
- Python stack supports reproducible end-to-end GIS workflows
- QGIS remains the fastest visual QA tool in the loop

References

-  Erik Westra.
Python Geospatial Development, 3rd ed.
Packt, 2017.
-  Rasterio developers.
Rasterio documentation.
<https://rasterio.readthedocs.io>
-  Sean Gillies et al.
Shapely documentation.
<https://shapely.readthedocs.io>
-  GeoPandas contributors.
GeoPandas documentation.
<https://geopandas.org>
-  pyproj developers.
pyproj documentation.
<https://pyproj4.github.io/pyproj/stable>

Thank You

Questions?

Next: run the two notebooks and validate one result in QGIS.

Roadmap Checkpoint

7 Backup Slides

Backup: CRS Decision Table

Goal	Recommended CRS	Why
Web map overlays	EPSG:3857	compatible with basemaps
Field data exchange	EPSG:4326	GPS-friendly global standard
Local distance/area	Local UTM zone	meter-based measurements
Regional equal-area stats	Albers Equal Area	area-preserving summaries

Backup: Clip Vectors to Raster Bounds

```
import geopandas as gpd
import rasterio
from shapely.geometry import box

with rasterio.open("scene.tif") as src:
    bounds_poly = box(*src.bounds)
    raster_crs = src.crs

gdf = gpd.read_file("labels.geojson").to_crs(raster_crs)
clip_box = gpd.GeoDataFrame(geometry=[bounds_poly], crs=raster_crs)
out = gpd.clip(gdf, clip_box)
out.to_file("labels_clipped.gpkg", driver="GPKG")
```

Backup: Common Failure Modes

- Lat/lon axis order confusion in manual transforms
- Mixing EPSG:4326 vectors with projected rasters
- Assuming shapefile preserves full schema metadata
- Writing outputs without explicit nodata declarations